SIMATS ENGINEERING

PROGRAMMING IN JAVA
ASSIGNMENT 4 — CO4

EVENT-DRIVEN GUI DESIGN

Student Name	D. Saketh
Register Number	192525018
Course	CSA09 — Programming in Java
JDK	OpenJDK 21
GitHub Repository	Pending creation

GitHub repository is not yet created. Replace the placeholder with the working URL before submission.

Task 1 — GUI Design Review (6 marks)

Diagnosis: The same listener mixes UI and payment logic; `==` compares String references; input is not validated; JDBC work and `Thread.sleep(3000)` block the event-dispatch thread; no user-facing success/error dialog; Receipt has no implementation; SQL concatenation risks injection; connection is not closed; exceptions are not handled; window closing is not configured.

Repair: FeeWindow gathers register number and amount and displays status. FeeService validates required fields and positive numeric amount. Each button has a focused listener. Production JDBC should use PreparedStatement, try-with-resources and SwingWorker for slow work.

Defence: Separating the service from the window allows service validation to be unit-tested without constructing a GUI. UI changes do not require rewriting payment rules, and payment logic can be reused.

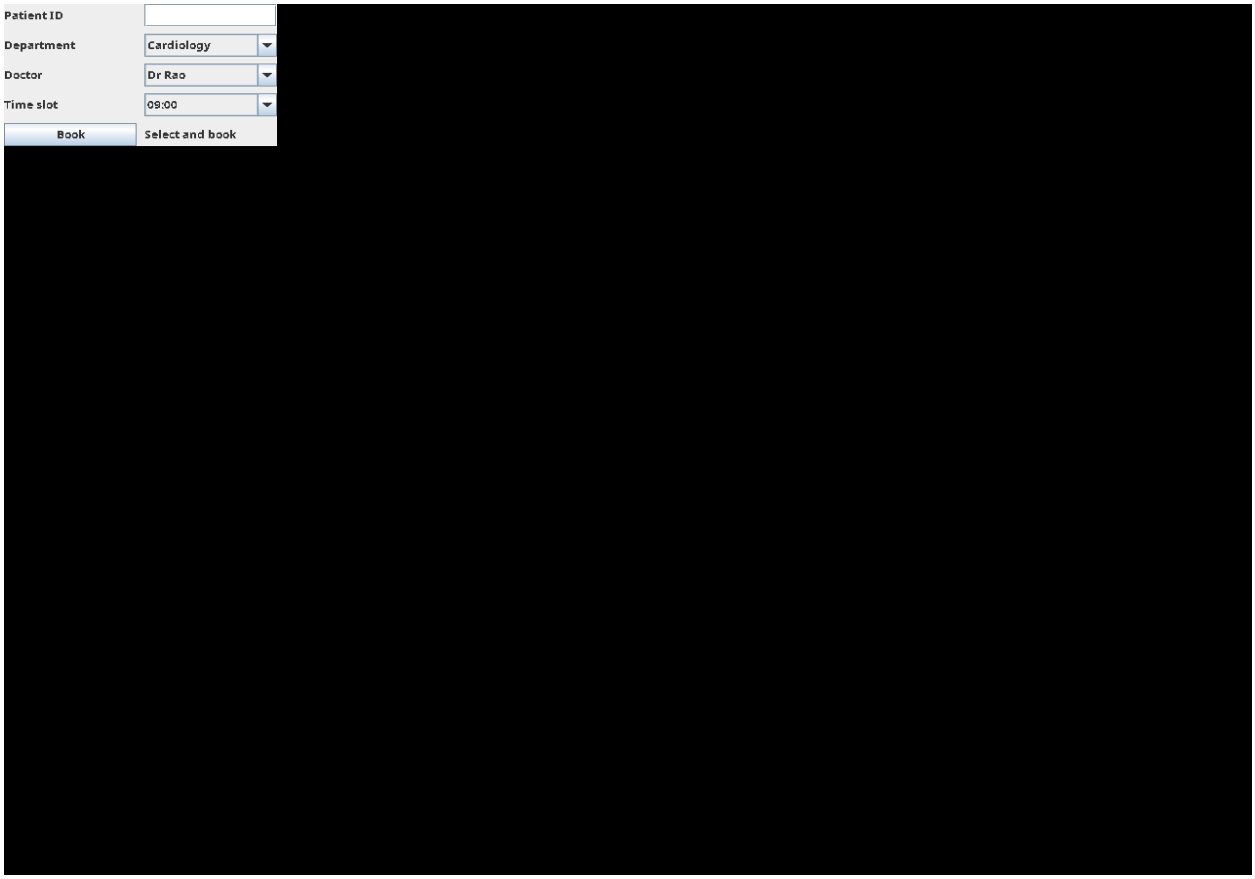
Task 2 — Hospital Appointment Desk (8 marks)

Event model: Patient ID JTextField → ActionEvent/ActionListener on Enter if enabled; department JComboBox → ItemEvent/ItemListener or ActionEvent/ActionListener refreshes doctors; Book JButton → ActionEvent/ActionListener validates and books; window close → WindowEvent/WindowListener asks confirmation.

Component hierarchy: JFrame → content JPanel (GridLayout) → patient JTextField, department JComboBox, doctor JComboBox, slot JComboBox, Book JButton. JOptionPane supplies confirmation/error dialogs.

Handler pseudocode: ON departmentChanged: clear doctor choices; read selected department; add doctors from Map>. ON book: validate patient ID, doctor and slot; if invalid show error; otherwise mark booking in progress, invoke service, show confirmation; clear progress flag.

Event trace: Construct frame → initialize components/listeners → pack/setVisible → event dispatch thread receives selection/action events → handler updates UI/service → windowClosing asks confirmation if booking is active → dispose. Unlike applet init/start/paint/stop/destroy lifecycle callbacks, desktop windows are driven by AWT events.



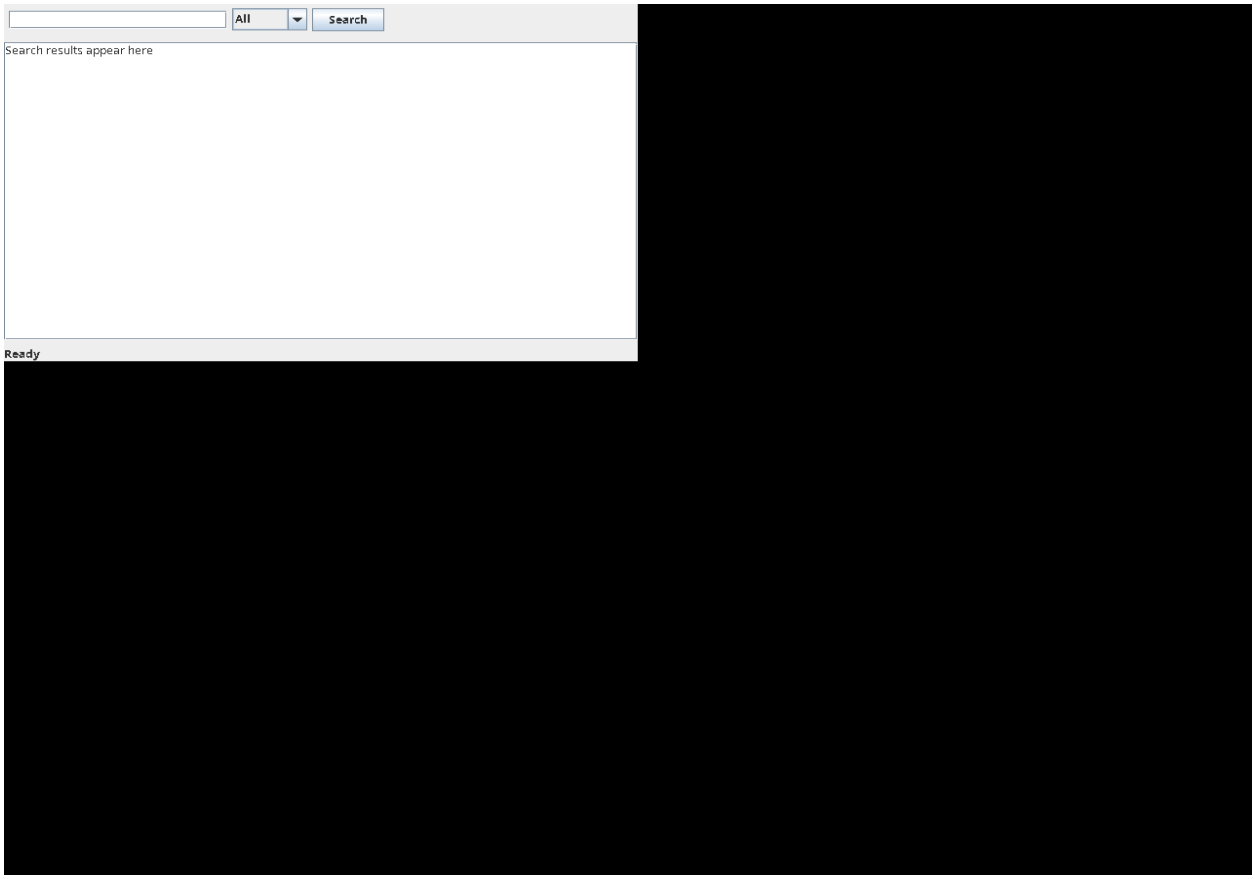
Task 3 — Campus Library Search Layout (9 marks)

Layout choices: BorderLayout for main frame: NORTH title, CENTER results, SOUTH navigation/status. FlowLayout for search row and bottom buttons. Results scroll pane occupies CENTER and expands on resize; fixed rows retain preferred sizes.

Nested structure: JFrame(BorderLayout) → title JLabel(NORTH); search JPanel(FlowLayout) in PAGE_START; JScrollPane(JTextArea)(CENTER); bottom JPanel(FlowLayout) in SOUTH.

Implementation and resize evidence: LibrarySearch.java implements the Swing screen. A live GUI screenshot is included. The three requested window-size screenshots should be captured on the student's desktop after resizing to small, medium and large sizes; a single automated launch capture is not claimed as all three.

Alternative: GridBagLayout offers flexible constraints but requires more setup. Absolute positioning is brittle on resize and font scaling. Nested layout managers add a small number of containers but adapt reliably.



Task 4 — Event-Handling Forensics (7 marks)

Defect: The method is misspelled `mouseclicked`; Java requires `mouseClicked`, so the listener method is never overridden. The label is never updated. Drawing directly with getGraphics() is temporary and disappears on repaint.

Failing interaction: Click to draw, minimize the window, then restore it. A repaint clears the transient drawing because the points were not retained.

Repair: The corrected DrawPad overrides mouseClicked, stores each Point, updates the click label and calls repaint(). paint() iterates through stored points and draws a 6×6 dot centered at each point.

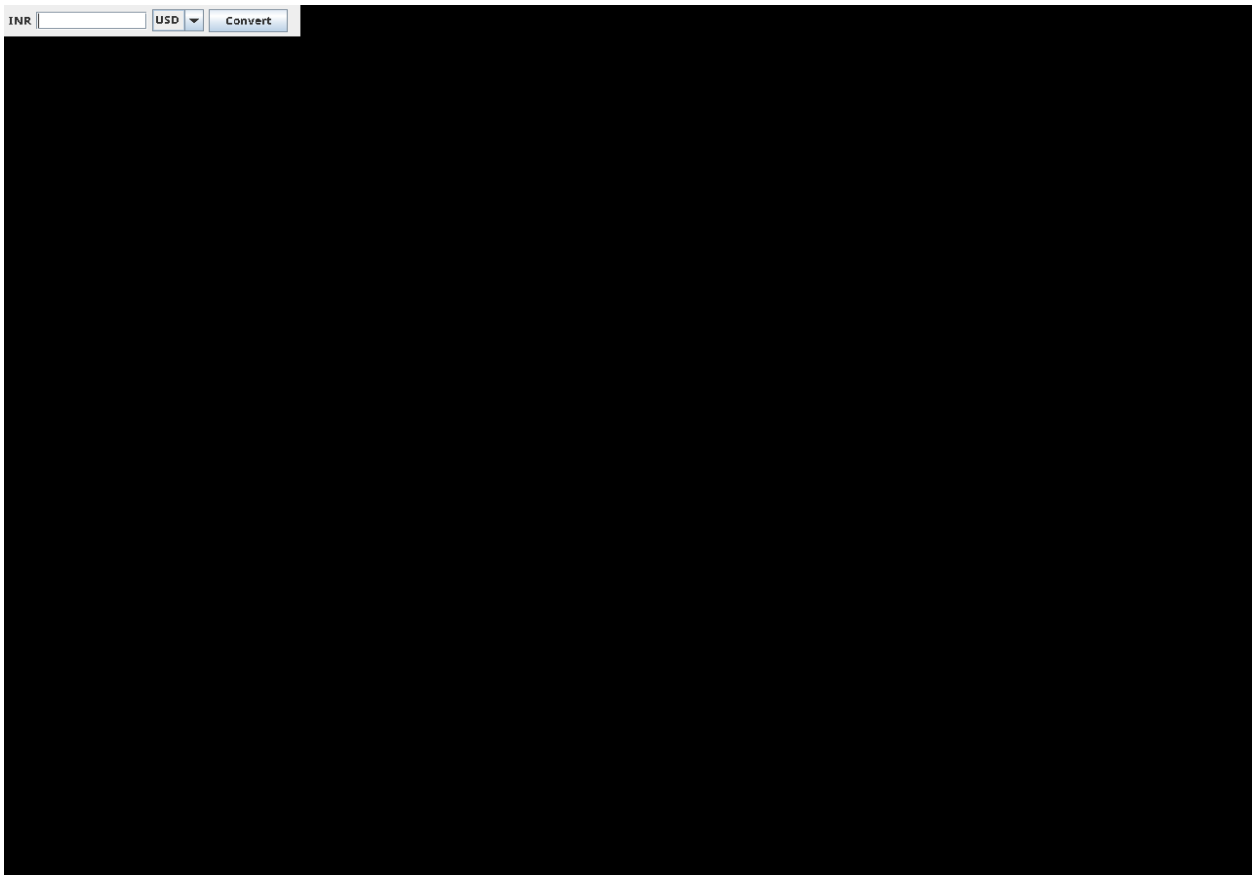
Prevention: @Override causes a compile-time error when the method signature does not match MouseListener/MouseAdapter.

Task 5 — Event-Handling Styles (7 marks)

Four styles: HandlerStyles.java demonstrates the frame implementing ActionListener, a named inner class, an anonymous inner class and a lambda. ConverterStyles.java provides the requested lambda-style converter with one listener shared by the button and Enter key.

Requirement change: To trigger on Enter, register the same ActionListener on the amount JTextField. To show a dialog, call JOptionPane.showMessageDialog after successful conversion or on invalid input.

Comparison: Frame listener is simple for tiny windows but couples the frame to the interface. Named inner classes are reusable/readable. Anonymous classes are verbose. Lambdas are concise for single-method interfaces. For a large application, use named actions or shared Action objects/method references for reuse.



Task 6 — Campus Event Registration App (13 marks)

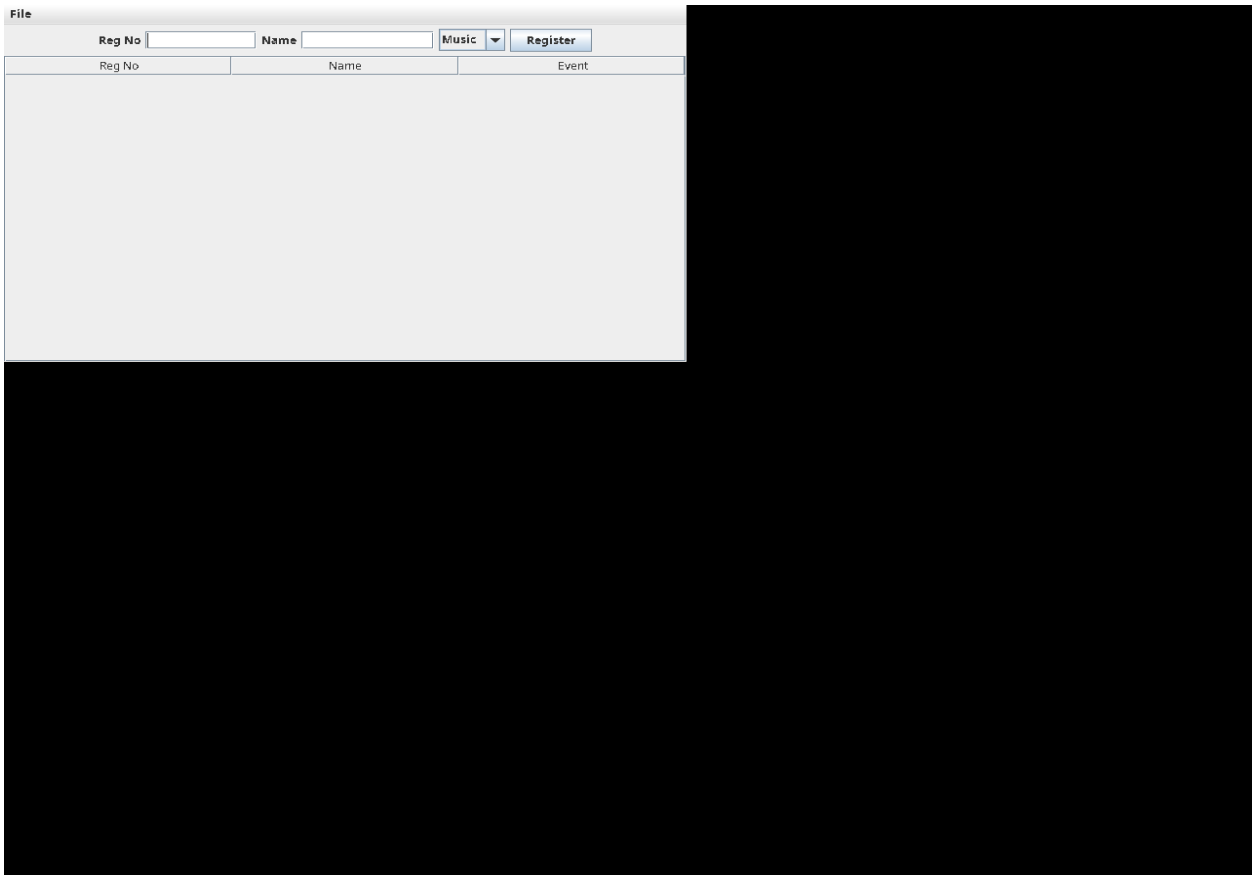
Requirements and layers: Student registers for events; committee views, filters and exports. Screens: registration form and registrations table. Data: student registration, event, timestamp. Rules: required fields, unique (reg_no,event_id), clear validation, responsive loading.

Architecture: EventRegistrationApp.java is a Swing prototype with JFrame, JMenuBar, form fields and JTable. It validates empty fields and rejects duplicate register/event pairs in its in-memory model. Production architecture should separate EventRegistrationFrame → RegistrationService → RegistrationDao (JDBC).

Responsiveness and persistence: Use SwingWorker for database reads/writes so JDBC does not block the event-dispatch thread. Enforce a UNIQUE(reg_no,event_id) constraint in MySQL; application-side checks alone cannot prevent concurrent duplicate submissions. Use PreparedStatement and try-with-resources.

Evidence limitation: The included GUI prototype runs and demonstrates local validation/duplicate detection. It does not connect to a configured MySQL server; a database-backed implementation, SQL script and live JDBC test still need database credentials/driver. No successful database insertion is claimed.

Paging alternative: Load one page using LIMIT/OFFSET in SwingWorker to limit UI memory to O(page size). Loading 100,000 rows at once requires O(n) row memory and can freeze the EDT if performed there; paged retrieval reduces retained rows and keeps the window responsive.


File
Reg No
Name
Music
Register
Reg No
Name
Event